

Bidirectional Interpolation for the Lambda-Calculus

Revisiting and Formalising Craig-Čubrić Interpolation

Meven Lennon-Bertrand 

Université Paris Cité, INRIA, CNRS, IRIF, F-75013 Paris, France

Alexis Saurin 

Université Paris Cité, CNRS, INRIA, IRIF, F-75013 Paris, France

Abstract

Craig's Interpolation theorem has a wide range of applications, from mathematical logic to computer science. Proof-theoretic techniques for establishing interpolation usually follow a method first introduced by Maehara for the Sequent Calculus and then adapted by Prawitz to Natural Deduction. The result can be strengthened to a proof-relevant version, taking proof terms into account: this was first established by Čubrić in the simply-typed lambda-calculus with sums and more recently in linear, classical and intuitionistic sequent calculi. We give a new proof of Čubrić's proof-relevant interpolation theorem by building on principles of bidirectional typing, and formalise it in ROCQ.

Keywords and phrases Craig Interpolation, Bidirectional Typing, Typed Lambda Calculus

Supplementary Material *Rocq Formalisation:* anonymous.4open.science/r/interpolation-stlc

1 Introduction

Craig-Lyndon Interpolation Craig's interpolation theorem [10, 11] expresses a desirable, though unsurprising, property of logical entailment: a logical entailment of B from A can be mediated through a logical statement that only uses the *common concepts* of A and B . More precisely, a *Craig Interpolant* I of $A \vdash B$ is a logical formula that is built using only the predicate symbols occurring both in A and B and such that $A \vdash I$ and $I \vdash B$. This was soon recognized as a central result of first-order logic [17, 53] with application in both mathematical logic [42, 4, 17, 52] (notably an elegant proof of Beth' definability theorem and of Robinson joint consistency theorem) and computer science [15, 51, 33] (with applications in database query planning or modularisation of model-checking), and numerous generalizations to extensions or restrictions of classical logic [52, 45, 40].

Proof-theoretic methods to establish Craig's theorem essentially rely on cut admissibility in the sequent calculus and on the subformula property [30, 40]. This very fact, providing well-structured methods [30, 40, 50, 45, 51] plays an important role when generalising interpolation to various logics as mentioned above, as well as in the existence of finer-grained notions of interpolations, most notably (i) Lyndon interpolation [29], which is sensitive to the polarity of predicate symbol occurrences, and (ii) uniform interpolation [39], in which, given a formula A , a single interpolant can be generated, uniformly, for a whole sub-language.

Proof-relevant Interpolation Considering the importance of proof-theoretic methods for interpolation, it is not so surprising that interpolation theorems can be given a proof-relevant refinement, as recently advocated by Saurin [43]. This means that not only can one factor the entailment through the interpolant formula, but also that the proofs from and to the interpolant, once composed, yield the original proof. Even more interesting is that proof-relevant interpolation was implicit already in Maehara's original results in the 60s.

This is exactly what is at stake in Čubrić's interpolation theorem [55, 56] for the simply typed λ -calculus with sums and empty types (STLC₊), a proof-relevant refinement of Prawitz's proof of interpolation [40] for natural deduction. This theorem essentially states that for every types A, B and every normal term t such that $x : A \vdash t : B$, there is a type

C , built from the atomic types occurring both in A and B , and terms $x: A \vdash u : C$ and $y: C \vdash v : B$ such that $(\lambda y.v) u \rightsquigarrow^* t$. Owing to the fact that STLC_+ is the internal language of bi-cartesian closed categories (biCCC), Čubrić moreover leverages his interpolation result to obtain categorical corollaries, including categorifications of Pitt’s results on Heyting algebras, the posetal counterpart to biCCCs.

Interpolation meets bidirectional typing Although very interesting, Čubrić’s original proof is... not exactly pretty. The core argument goes by an induction on the size of terms, gradually replacing subterms by variables. Moreover, the one case dismissed in the proof as “the same as the previous” [56, Subcase 6.3.2], is in fact not at all similar. However, we can discern more structure in the proof, as it essentially follows a characterisation of normal forms in STLC_+ going back to Prawitz. We have made progress since Prawitz, and it is nowadays routine [1, 18] to re-cast this characterisation as an explicit, inductive description of normal forms, mutually with neutrals, those normal forms consisting of stuck eliminations. Our new proof thus goes by a direct, mutual induction on this inductive characterisation.

A difficulty is that, in the presence of positive type formers (sums and the empty type), β -normal forms are not enough: interpolation works well for terms/proofs satisfying the subformula property, which in STLC_+ mere β -normal forms do not have. We hence need to go further and normalise with respect to *commuting conversions* [24], which exchange certain eliminations to recover a class of normal forms with a good subformula property.

Surprisingly, our inductive normal forms are naturally interpreted in terms of bidirectional typing [38, 16]. Although this connection has been drawn before, it is often believed that it breaks for sums. We think this is a confusion stemming from the fact that STLC_+ has two notions of normal forms. The first is characterised as having the subformula property, which enables our interpolation proof. But, contrarily to the situation without sums, these do not give unique representative of equivalence classes for biCCC’s equational theory, for which one needs a second, stronger notion [44]. Since bidirectional typing has all to do with information flow, it naturally coincides with the *first* kind of normal forms, although it does not correspond with the second any more. We believe this relation between bidirectionality and the subformula property deserves more publicity, which we hope this paper can contribute.

Contributions We develop and formalise, in the ROCQ proof assistant, the meta-theory of the λ -calculus with sums, including the first formalised proof of normalisation for a calculus with commuting conversions; we relate normal forms to bidirectional typing, and use this in a new proof of proof-relevant interpolation.

Notations We work in dependent type theory, but to avoid confusion between its judgments and those of the object language, we denote meta-level typing as $\in$ and use $\mathbb{P}\text{rop}$ and $\mathbb{S}\text{et}$ for meta-level universes of propositions and types — of which we need only one. We use colours to suggest information: a term being **normal** or **neutral**, or an object containing **source**, **target** or **shared** vocabulary. The same name with different colours denotes *the same object* (with updated knowledge). Throughout the text,  indicates links to the (anonymised) code.

2 The simply-typed λ -calculus with sums

2.1 Syntax

Types and terms We start with the simply-typed λ -calculus, hereafter abbreviated as STLC_+ , with function types $\rightarrow$, product types $\times$, sum types $+$, and unit and empty types

$$\begin{array}{c}
\boxed{\vdash_{\mathcal{B}} \Gamma} \quad \text{Contexts } \Gamma, \Delta, \dots \in \text{Ctx}_{\mathcal{B}} \\
\frac{}{\vdash \cdot} \quad \frac{\vdash \Gamma \quad \vdash A}{\vdash \Gamma.(x:A)} \\
\\
\boxed{\vdash_{\mathcal{B}} A} \quad \text{Types } A, B, \dots \in \text{Ty}_{\mathcal{B}} \\
\frac{b \in \mathcal{B}}{\vdash_{\mathcal{B}} b} \quad \frac{}{\vdash \top} \quad \frac{\vdash A \quad \vdash B}{\vdash A \times B} \quad \frac{\vdash A \quad \vdash B}{\vdash A \rightarrow B} \quad \frac{}{\vdash \perp} \quad \frac{\vdash A \quad \vdash B}{\vdash A + B} \\
\\
\boxed{\Gamma \vdash_{\mathcal{L}} t : A} \quad \text{Terms } t, u, \dots \in \text{Tm}_{\mathcal{L}}(\Gamma, A) \\
\text{CST} \frac{c \in \mathcal{C}}{\Gamma \vdash_{(\mathcal{B}, \mathcal{C}, \text{ty})} c : \text{ty}(c)} \quad \text{VAR} \frac{(x:A) \in \Gamma}{\Gamma \vdash x : A} \quad \text{STAR} \frac{}{\Gamma \vdash * : \top} \quad \text{PAIR} \frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B}{\Gamma \vdash (t, u) : A \times B} \\
\\
\text{PROJ} \frac{\Gamma \vdash p : A_1 \times A_2}{\Gamma \vdash p.i : A_i} \quad \text{LAM} \frac{\Gamma.(x:A) \vdash t : B}{\Gamma \vdash \lambda x.t : A \rightarrow B} \quad \text{APP} \frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \\
\\
\text{RAISE} \frac{\Gamma \vdash t : \perp}{\Gamma \vdash \text{raise } t : A} \quad \text{INL} \frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl } a : A + B} \quad \text{INR} \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr } b : A + B} \\
\\
\text{IF} \frac{\Gamma \vdash s : A + B \quad \Gamma.(x:A) \vdash b_l : T \quad \Gamma.(x:B) \vdash b_r : T}{\Gamma \vdash \text{if}(s, b_l, b_r) : T}
\end{array}$$

■ **Figure 1** Types and terms of STLC₊ with respect to a language $\mathcal{L} = (\mathcal{B}, \mathcal{C}, \text{ty})$

$\top$ and $\perp$. To be able to state an interesting version of interpolation, we also include an unspecified set of base types. To make the result more interesting, we likewise include a language of constants.

► **Definition 1** (🎮 Language). A language $\mathcal{L}$ consists of:

- a set of base types $\mathcal{B} \in \text{Set}$;
- a set $\mathcal{C} \in \text{Set}$ of constants;
- types for each constant, i.e. a function $\text{ty} \in \mathcal{C} \rightarrow \text{Ty}_{\mathcal{B}}$.¹

► **Definition 2** (🎮 Types and terms). The definition of types and terms is given in Fig. 1. We decompose types into the negative types ($\rightarrow$, $\times$ and $\top$) and the positive types (base types, $+$ and $\perp$). If $\mathcal{L} = (\mathcal{B}, \mathcal{C}, \text{ty})$ we write $\text{Ty}_{\mathcal{L}}$ for $\text{Ty}_{\mathcal{B}}$, and similarly for contexts. When it is not particularly relevant, we omit the language subscript.

Substitutions We work with parallel substitutions, the action of which we denote $t[\sigma]$. Although we use variable names for exposition, we formally think of them in the de Bruijn style [14], and formalise them this way in ROCQ. Accordingly, the substitution calculus we use is essentially name-free. We include the primitives we use in the rest of the text in Fig. 2. The typing judgment $\Delta \vdash \sigma : \Gamma$ means that σ takes a variable $(x:A) \in \Gamma$ to a term $\sigma(x) \in \text{Tm}(\Delta, A)$. The primitive's types should already be informative, but let us

¹ $\text{Ty}_{\mathcal{B}}$ is the set of types generated by $\mathcal{B}$, to be defined in Definition 2; formally these two definitions should be intertwined.

$\boxed{\Delta \vdash \sigma : \Gamma}$ Substitutions $\sigma, \tau, \dots \in \text{Sub}_{\mathcal{L}}(\Delta, \Gamma)$

$$\text{WK} \frac{}{\Gamma.(x:A) \vdash \uparrow : \Gamma} \quad \text{CONS} \frac{\Delta \vdash \sigma : \Gamma \quad \Delta \vdash t : A}{\Delta \vdash \sigma, t : \Gamma.(x:A)} \quad \text{ONE} \frac{\Gamma \vdash t : A}{\Gamma \vdash t.. : \Gamma.(x:A)}$$

■ **Figure 2** 🐼 Substitutions and their typing (excerpt)

describe them succinctly: $\uparrow$ drops the last variable in context, and maps all other variables to themselves;² σ, t extends a substitution σ by mapping the last variable in context to t ; finally $t..$ maps the last variable to t and acts as the identity otherwise. The following substitution rule is admissible (🐼):

$$\text{SUBST} \frac{\Delta \vdash \sigma : \Gamma \quad \Gamma \vdash \sigma : A}{\Delta \vdash t[\sigma] : A}$$

Renamings, denoted $\text{Ren}(\Delta, \Gamma)$, are the substitutions that map variables to variables.

2.2 Reduction and equations

The language defined in Fig. 1, quotiented by the adequate equational theory, is the internal language of bi-cartesian closed categories (biCCC), *i.e.* with finite products, finite coproducts, and exponentials. Normalisation for this equational theory is unwieldy due to η laws for positive types (see Scherer [44]). Fortunately, we can get away with normalisation for a less powerful reduction containing only the rules known as *commuting conversions* [24]. This is because normal forms for this reduction have a good enough subformula property, even though they are not unique representatives for the equational theory of biCCCs.

Reduction To express commuting conversions, we use *eliminations*, given by the following grammar (🐼): $e ::= \square \mid t \mid \square.i \mid \text{if}(\square, t, t)$ The *zip* of a term against an elimination $\langle t \mid e \rangle$ is defined in the obvious way (🐼).

Reductions are of two kinds: the usual β -reductions, corresponding to computation—applied abstraction, projection of a pair, and branching on a constructor—and commuting conversions, which push an elimination across another one of a positive type (*i.e.* $\perp$ or a sum). In the case of $\perp$, the elimination gets completely removed, as it is unnecessary: a proof of $\perp$ can be used to inhabit any type, there is no need to further eliminate it. In the case of the sum, the elimination gets duplicated in each branch of the *if*.

► **Definition 3** (🐼 Reductions). One-step reduction $\rightsquigarrow$ is the congruent closure of the rules given in Fig. 3. Reduction $\rightsquigarrow^*$ is the reflexive, transitive closure of one-step reduction.

► **Theorem 4** (🐼 Confluence). *Reduction is confluent: if $t \rightsquigarrow^* u$ and $t \rightsquigarrow^* u'$, then there exists v such that $u \rightsquigarrow^* v$ and $u' \rightsquigarrow^* v$.*

Since confluence is rather standard [23] yet tedious to prove, and orthogonal to our focus, we do not formally prove it in ROCQ, but only provide a pen-and-paper proof sketch.³

² In named syntax this is a no-op, but we include it as it changes the context in which a term is considered.

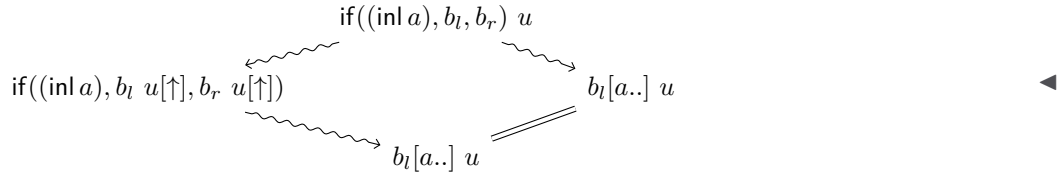
³ Surprisingly, despite our best efforts we were not able to find a detailed proof of that fact.

$t \rightsquigarrow u$ One-step reduction

$$\begin{array}{c}
\text{PAIR}_1\beta \frac{}{(t, u).1 \rightsquigarrow t} \quad \text{PAIR}_2\beta \frac{}{(t, u).2 \rightsquigarrow u} \quad \text{FUN}\beta \frac{}{(\lambda x.t) u \rightsquigarrow t[u..]} \\
\\
\text{INL}\beta \frac{}{\text{if}(\text{inl } a, b_l, b_r) \rightsquigarrow b_l[a..]} \quad \text{INR}\beta \frac{}{\text{if}(\text{inr } b, b_l, b_r) \rightsquigarrow b_r[b..]} \\
\\
\text{COMMRais} \frac{}{\langle \text{raise } t \mid e \rangle \rightsquigarrow \text{raise } t} \quad \text{COMMIf} \frac{}{\langle \text{if}(s, b_l, b_r) \mid e \rangle \rightsquigarrow \text{if}(s, \langle b_l \mid e \rangle, \langle b_r \mid e \rangle)}
\end{array}$$

■ **Figure 3**  Reduction (congruence rules omitted)

Proof. We use the Hindley-Rosen lemma [3, Prop. 3.3.5], by which it suffices to show that 1. β -reduction is confluent 2. commuting conversion is confluent 3. the two commute. For the first, since there is no critical pair Takahashi’s variant of parallel reduction à la Tait–Martin-Löf [48, 49] does the trick. For the second, strong normalisation of commuting conversion can be shown by a simple $\mathbb{N}$ -valued measure () , so by Newman’s lemma [3, 3.1.42], it suffices to show local confluence () . This is painful due to the many critical pairs, so we have checked this with the automated tool CSI^{HO} [37]. Finally, commutation again does not need normalisation because critical pairs between β and commuting conversion are all akin to the following, which can be closed in one step.



Equational theories To formally relate reduction to biCCCs, we can define () a notion of conversion $\Gamma \vdash t \equiv t' : A$ capturing equality of morphisms in a biCCC as the least congruent equivalence relation containing β and η laws, encoding the universal property of each type former. These are strictly stronger than the commuting conversions of Fig. 3; indeed we have

$$x : A + A, y : A + A, z : A \vdash \text{if}(x, \text{if}(y, z, z), \text{if}(y, z, z)) \equiv \text{if}(y, \text{if}(x, z, z), \text{if}(x, z, z)) : A$$

by η for sums, but the two terms are not related by commuting conversions alone.

This is not an issue, as our interpolation proof only uses conversion in covariant positions, so we show stronger theorems using reduction, and can weaken them to conversion, owing to:

► **Lemma 5** ( Subject reduction). *If $\Gamma \vdash t : A$ and $t \rightsquigarrow^* u$ then $\Gamma \vdash t \equiv u : A$.*

We actually go further and define a notion of *theory* () , which extends the specification of a language with a set of arbitrary equations that are added to its conversion. The proofs are essentially unchanged, but this means that our interpolation can be leveraged to prove general properties of biCCCs by viewing STLC as their internal language, although we do not explore this further here, and refer to Čubrić [56, 55] for details.

3 Bidirectional typing, normal forms and the subformula property

In logic, a key characteristic of derivations is the *subformula property*, satisfied when every formula appearing in the derivation is a subformula of some formula in the conclusion judgment — hypothesis or conclusion. This does not hold of arbitrary derivations, but is satisfied by those which are sufficiently normalised, *i.e.* cut-free proofs [24]. In a derivation satisfying the subformula property, no formula is ever “invented”, a powerful structural property with many consequences. Interpolation is for instance typically proven for cut-free derivations. One then obtains properties for all provable statements by a combination of cut elimination, by which every proof is equal to a cut-free one, and an analysis of cut-free proofs, which can exploit its more constrained structure, including the subformula property.

For a type theorist, this might seem a bit mysterious. Cut elimination, by the Curry-Howard correspondence, is well-known to coincide with reduction of terms viewed as proof witnesses. However, what does the subformula property amount to? This is much less known, but it can be very helpfully linked to *bidirectional typing*.

Bidirectional typing, which originates from the implementation of type-checking algorithms [9, 38], typing is decomposed into the mutually defined *inference* and *checking* judgments. The operational intuition is that in inference the type is an output,⁴ while in checking it is an input.⁵ This attention to the flow of information is, in a sense, what the subformula property also intends to capture: in both cases, the intuition is that type information always comes from somewhere and is never invented.

With this view it is perhaps less surprising that bidirectional typing can be used to capture normal forms, since these correspond to cut-free proofs which have a good subformula property. This is, however, a remark with powerful consequences: because bidirectional type systems have a nice, inductive structure, they give a good basis on which to build clean proofs of theorems that rely on the subformula property, such as interpolation. The remainder of this article is dedicated to delivering that idea: the rest of this section sets up the bidirectional characterisation of normal forms, while Sec. 4 derives the interpolation result.

3.1 A bidirectional characterisation of normal forms

Our bidirectional type system is given in Fig. 4, and was inspired by the normal forms of Abel and Sattler [2] —suitably modified to not enforce η -long forms. It can be interpreted in two ways.

In the bidirectional mindset, the rules can be read as a typing algorithm for a fragment of STLC_+ . Rules for the inference judgment $\Gamma \vdash t \triangleright A$ correspond to settings where it is feasible to infer the type A from t , by inferring that of a carefully chosen subterm (in the case of a negative elimination APP , PROJ_2) or directly reading it from the local (VAR) or global (CST) context. Conversely, rules for checking $\Gamma \vdash t \triangleleft A$ are those where it is better to check that a term has a given type. This encompasses constructors (STAR , PAIR , LAM , INL , INR), where the type is decomposed into parts which are used to check the subterms;⁶ any inferring term, which we can check provided the inferred and checked types coincides (DEMOTE); and more interestingly, the elimination for positive types (RAISE , IF). It is rather clear that raise cannot be reasonably inferring: how are we supposed to guess its type? For

⁴ *I.e.* the question is “What is the type of this term?”.

⁵ And the question is “Does this term have this type?”.

⁶ One could wonder why STAR is not inferring. While possible, it is more uniform to treat $\star$ as a nullary version of pairing. Anyway, since it can only appear in a checking position this makes little difference.

$\boxed{\Gamma \vdash t \triangleleft A}$ Checking/normal forms $t, u, \dots \in \text{Nf}_{\mathcal{L}}(\Gamma, A)$

$$\begin{array}{c}
\text{STAR} \frac{}{\Gamma \vdash * \triangleleft \top} \quad \text{PAIR} \frac{\Gamma \vdash t \triangleleft A \quad \Gamma \vdash u \triangleleft B}{\Gamma \vdash (t, u) \triangleleft A \times B} \quad \text{LAM} \frac{\Gamma, x: A \vdash t \triangleleft B}{\Gamma \vdash \lambda x. t \triangleleft A \rightarrow B} \\
\\
\text{INL} \frac{\Gamma \vdash a \triangleleft A}{\Gamma \vdash \text{inl } a \triangleleft A + B} \quad \text{INR} \frac{\Gamma \vdash b \triangleleft B}{\Gamma \vdash \text{inr } b \triangleleft A + B} \quad \text{DEMOTE} \frac{\Gamma \vdash t \triangleright A \quad A = B}{\Gamma \vdash \underline{t} \triangleleft B} \\
\\
\text{RAISE} \frac{\Gamma \vdash t \triangleright \perp}{\Gamma \vdash \text{raise } t \triangleleft A} \quad \text{IF} \frac{\Gamma \vdash s \triangleright A + B \quad \Gamma.(x: A) \vdash b_l \triangleleft T \quad \Gamma.(x: B) \vdash b_r \triangleleft T}{\Gamma \vdash \text{if}(s, b_l, b_r) \triangleleft T}
\end{array}$$

$\boxed{\Gamma \vdash t \triangleright A}$ Infering/neutral forms $t, u, \dots \in \text{Ne}_{\mathcal{L}}(\Gamma, A)$

$$\begin{array}{c}
\text{CST} \frac{c \in \mathcal{C}}{\Gamma \vdash_{\mathcal{L}} c \triangleright \text{ty}(c)} \quad \text{VAR} \frac{(x: A) \in \Gamma}{\Gamma \vdash x \triangleright A} \\
\\
\text{PROJ}_1 \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_1(p) \triangleright A} \quad \text{PROJ}_2 \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_2(p) \triangleright B} \quad \text{APP} \frac{\Gamma \vdash t \triangleright A \rightarrow B \quad \Gamma \vdash u \triangleleft A}{\Gamma \vdash t u \triangleright B}
\end{array}$$

■ **Figure 4** 🎮 Bidirectional typing for the STLC₊, with respect to a language $\mathcal{L} = (\mathcal{B}, \mathcal{C}, \text{ty})$

if, we could infer the type of the branches, comparing them, and if they agree infer that type for the whole pattern-matching. However, our version ends up being better behaved.

In this view, the bidirectional type system arises from the will to type annotation-free terms. To do so, we can only rely on type information which is already available, either in the context, or in the type in the case of checking. This means that all the type information should appear in these, which, to reiterate, is exactly what the subformula property captures!

The other approach views the $\Gamma \vdash t \triangleleft A$ judgment as characterising *normal forms*, those which do not reduce. When inductively characterising normal forms, it is necessary to mutually define *neutral forms*, those which can be used as the main argument of an elimination without creating redexes. Usually, any elimination of a neutral is again a neutral. But with commuting conversions this will not do, as an eliminator on top of raise or if reduces. So in that view too, it makes sense to classify these as normal forms but not neutrals.

We prove that this adequately captures well-typed terms that do not reduce, *i.e.* that our inductive characterisation indeed captures normal forms in the more traditional sense.

► **Lemma 6** (🎮 Normality). *If $\Gamma \vdash t \triangleleft A$, then $\Gamma \vdash t : A$ and there is no t' such that $t \rightsquigarrow t'$.*

► **Theorem 7** (🎮 Progress). *If $\Gamma \vdash t : A$, then either there is t' such that $t \rightsquigarrow t'$, or $\Gamma \vdash t \triangleleft A$.*

3.2 Normalisation for STLC₊

In Sec. 4, we prove interpolation for normal forms only. To apply it more generally, we show that every well-typed term reduces to a normal form. We give a proof by logical relations, heavily inspired by Abel and Sattler [2], although compared to their normalisation-by-evaluation approach we are able to obtain a result about reduction rather than conversion.

In a logical relation argument, the key definition is that of values, which goes by induction on types. As is standard, values at negative types are normal forms whose observations are

reducible: a value at a pair type has reducible projections, and a value at function type takes value inputs to reducible outputs. To describe values at positive types, we must take into account eliminations of $+$ and $\perp$, which are not neutrals. This leads us to introduce a notion of covering, capturing terms with the shape of a case tree, with nodes containing eliminations of neutrals at positive types, and leaves satisfying a given predicate (in a suitably extended context). It echoes the use of the free cover monad in *op. cit.* [2].

► **Definition 8** (🔒 Covering). A term $t \in \mathbf{Tm}(\Delta, A)$ is covered by the predicate $F \in (\Gamma \in \mathbf{Ctx}) \rightarrow \mathbf{Tm}(\Gamma, A) \rightarrow \mathbb{P}\mathbf{rop}$ if either:

- $F(t)$ holds;
 - $t = \text{raise } n$ with $\Gamma \vdash n \triangleright \perp$;
 - $t = \text{if}(n, b_l, b_r)$, with $\Gamma \vdash n \triangleright A + B$, and b_l, b_r are covered, resp. in $\Gamma.(x:A)$ and $\Gamma.(x:B)$.
- We say that a term $t \in \mathbf{Tm}(\Gamma, A)$ is “covered by neutrals” if it is covered by the predicate $F(\Delta, u) ::= \Delta \vdash u \triangleright A$, and similarly for other predicates on terms.

► **Definition 9** (🔒 Values). A raw value at a type T is a term $t \in \mathbf{Tm}(\Gamma, T)$ such that:

- if T is a base type or $\perp$ and t is neutral;
- T is $\top$ and t is either neutral or $\star$;
- T is $A + B$ and t is either neutral, $\text{inl}(a)$ with a a value of type A in Γ , or $\text{inr}(b)$ with b a value of type B in Γ .

A value at type T is a term $v \in \mathbf{Tm}(\Gamma, T)$ such that:

- if T is a positive type (base type, $\perp$, $A + B$), v is covered by raw values;
- if T is $\top$, v is a normal form;
- if T is $A_1 \times A_2$, v is a normal form and v_i is reducible at A_i ;
- if T is $A \rightarrow B$, v is a normal form and for any context Δ , renaming $\Delta \vdash \rho : \Gamma$, and value $u \in \mathbf{Tm}(\Delta, A)$, $v[\rho] u$ is reducible.

► **Definition 10** (🔒 Reducibility). A term t is reducible if there is a value v such that $t \rightsquigarrow^* v$.

We extend this to substitutions: a value substitution is a substitution $\sigma \in \mathbf{Sub}(\Delta, \Gamma)$ mapping each variable $(x:A) \in \Gamma$ to a value $\sigma(x)$ at A . We use this to define semantic typing.

► **Definition 11** (🔒 Semantic typing). A term t is semantically well-typed, written $\Gamma \Vdash t : A$, if for any value substitution $\sigma \in \mathbf{Sub}(\Delta, \Gamma)$, $t[\sigma]$ is reducible at A in Δ .

Before we can get on to proving the semantic counterparts to the typing rules in Fig. 1, we first need a handful of properties of covers, values and reducible terms.

► **Lemma 12** (🔒 Reification). Any value is a normal form.

► **Lemma 13** (🔒 Reflection). Any neutral is a value.

These lemmas are named by analogy with the reify/reflect functions of normalisation by evaluation. The lemmas are both proven by induction on types. For the first, the key ingredient is that a term covered by normal forms is again a normal form. We need reification in reflection, to handle reducibility at function types where to conclude that applying the neutral to a value yields a neutral we need to turn that value into a normal form.

Next, we show that a term covered by reducible terms is reducible. At positive types, it follows from the next lemma, which can be read as the multiplication of a covering monad.

► **Lemma 14** (🔒 Multiplication). A term covered by terms covered by F is covered by F .

At negative types, however, things are not that simple. Consider the function type: we assume that at each leaf of the cover we have terms which, when applied to a value, yield a reducible result, and need to deduce that the whole covered term is itself reducible, that is, yields reducible result when applied to a value. The first ingredient is the following lemma, which pushes application to the leaves of a cover by virtue of commuting conversions. We omit similar lemmas for the other eliminations.

► **Lemma 15** (♣ Covered application). *If $f \in \text{Tm}(\Gamma, A \rightarrow B)$ is covered by F and $u \in \text{Tm}(\Gamma, A)$, then $f \ u \rightsquigarrow^* t$ for some t which is covered by the predicate*

$$\Delta, t' \mapsto \exists(\rho: \text{Ren}(\Delta, \Gamma))(f': \text{Tm}(\Delta, A \rightarrow B)).(t' = f' \ u[\rho]) \wedge (F \ f')$$

Because pushing argument to the leaves introduces extra variables in the context, we must also know that values are preserved by context extension. This is what necessitates the Kripke-style quantification over renamings in the definition of values at function types. For the category theorist, this should be unsurprising: for values to form a presheaf over renamings, the definition of the semantic function type mirrors the presheaf exponential.

► **Lemma 16** (♣). *If v is a value of type A in Γ and $\rho \in \text{Ren}(\Delta, \Gamma)$, then $v[\rho]$ is a value.*

Putting these together, we finally obtain:

► **Lemma 17** (♣ Reducible cover). *A term covered by reducible terms is reducible.*

With this in hand, the rest of the proof is history, we let interested reader consult the formalisation. An interesting point to note, though, is that the analogue of Lemma 15 for raise and if is not used to prove Lemma 17, but to show the semantic typing rule for the eliminators: owing to the polarity of the types, the work for negative types is for constructors—where we need Lemma 17—, while eliminations are the hard part for positive types.

► **Theorem 18** (♣ Fundamental lemma). *If $\Gamma \vdash t : A$ then $\Gamma \Vdash t : A$.*

From Lemma 13 we get that variables are values at all types, and thus that the identity substitution is a value substitution. Hence, we conclude

► **Corollary 19** (♣ Normalisation). *Any well-typed term reduces to a normal form.*

4 Interpolation

With our characterisation of normal forms in hand, we can move to the interpolation result. Following Čubrić [56], we give a polarised version in the fashion of Lyndon interpolation [29], and thus start with a polarised notion of vocabulary.

► **Definition 20** (♣ Vocabulary). *Let $A \in \text{Ty}_{\mathcal{B}}$. We define $A^+ \subseteq \mathcal{B}$ (resp. A^-), the positive (resp. negative) vocabulary of A by induction, with $\overline{+} := -$ (resp. $\overline{-} := +$):*

$$\begin{aligned} b^+ &:= \{b\} & \top^p &:= \emptyset & (A + B)^p &:= A^p \cup B^p & (A \rightarrow B)^p &:= A^{\overline{p}} \cup B^p \\ b^- &:= \emptyset & \perp^p &:= \emptyset & (A \times B)^p &:= A^p \cup B^p \end{aligned}$$

This is lifted pointwise to contexts as $(\Gamma.(x:A))^p := \Gamma^p \cup A^p$.

Although stating the final theorem does not need it, in the proof by induction it is necessary to keep track of which variables in the context are “coming from” the conclusion, and thus we need to maintain a context partition in the following sense.

► **Definition 21** (Context partition). Context partition $\Gamma = \Gamma_1 \cup \Gamma_2$ is defined by

$$\frac{}{\cdot = \cdot \cup \cdot} \quad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x:A) = \Gamma_1.(x:A) \cup \Gamma_2} \quad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x:A) = \Gamma_1 \cup \Gamma_2.(x:A)}$$

In a named setting this is a mere relation: as long as we enforce that variable names in contexts are unique, they tell us how the partitioning is done. In the formalisation, where we use a nameless syntax, context partition becomes structure: there are two proofs of $A.A = A \cup A$, respectively corresponding, in a named setting, to $(x:A).(y:A) = (x:A) \cup (y:A)$ and $(x:A).(y:A) = (y:A) \cup (x:A)$. We spare the reader the substitution calculus arising from this, *e.g.* the fact that if $\Gamma = \Gamma_1 \cup \Gamma_2$ then there is a renaming $\Gamma \vdash \rho : \Gamma_1$.

4.1 Constant-free theorem

Our first theorem is constant-free: for this section we fix a language $\mathcal{L} = (\mathcal{B}, \emptyset, _)$.

► **Theorem 22** (Interpolation, constant-free). Assume $\Gamma \vdash_{\mathcal{L}} t : T$, then there exists a type M and terms $\Gamma \vdash l : M$ and $(x:M) \vdash r : T$ such that

- M uses only the common vocabulary of Γ and T : for any $p \in \{+, -\}$ $M^p \subseteq \Gamma^p \cap T^p$;
- t is the composition of r and l : $\Gamma \vdash t \equiv r[l.] : T$.

Since by normalisation every term is equal to a normal form, it is enough to show the following theorem for normal forms, and specialize it to the trivial partition $\Gamma = \Gamma \cup \cdot$.

► **Theorem 23** (Interpolation, inductive statement). Assume a partitioned context $\Gamma = \Gamma_s \cup \Gamma_t$. Given any normal form $t \in \text{Nf}(\Gamma, T)$, there is a type M and terms $l \in \text{Tm}(\Gamma_s, M)$ and $r \in \text{Tm}(\Gamma_t.(x:M), T)$ such that $\Gamma \vdash r[l.] \equiv t : T$ and for all p , $M^p \subseteq \Gamma_s^p \cap (\Gamma_t^{-p} \cup T^p)$. Moreover, given any neutral term $t \in \text{Ne}(\Gamma, T)$, either

1. “ T is a source type”, *i.e.* $T^p \subseteq \Gamma_s^p$, and we have a type M and terms $l \in \text{Tm}(\Gamma_t, M)$ and $r \in \text{Tm}(\Gamma_s.(x:M), T)$ such that $\Gamma \vdash r[l.] \equiv t : T$ and $M^p \subseteq \Gamma_s^{-p} \cap \Gamma_t^p$.
2. or “ T is a target type”, *i.e.* $T^p \subseteq \Gamma_t^p$, and we have a type M and terms $l \in \text{Tm}(\Gamma_s, M)$ and $r \in \text{Tm}(\Gamma_t.(x:M), T)$ such that $\Gamma \vdash r[l.] \equiv t : T$ and $M^p \subseteq \Gamma_s^p \cap \Gamma_t^{-p}$.

The statement for normal forms is essentially Čubrić’s [56, Lemma 3.7]. It is a generalisation of the interpolation statement, using the partition to account for some type information initially in the target type moving in the context, *e.g.* in the case of a λ -abstraction. The counterpart for neutrals, however, is new: the original proof did not use neutrals at all, and instead, went by strong induction on the size of terms, replacing subterms by variables. Our reasoning by mutual induction is much more direct, and also much easier to formalise!

Note that in case 1 the roles of the source and target context are exchanged in the typing of the interpolants compared to the statement for normal forms. It might make sense to push this distinction to bidirectional typing, and have a refined neutral judgment that also infers from which side of the context split its main variable comes. This might slightly clarify the present proof, but since it does not drastically affect it we will not explore it further.

Proof. As expected, the proof goes by induction on the definition of **Ne** / **Nf**.

Most **Nf** cases are rather direct, we only show **LAM**, cases for **PAIR**, **INL**, **INR** are similar.

LAM: We extend the *target* context Γ_t with the extra variable (this is why we need the formulation with a context partition!) and by induction hypothesis, we obtain $l' \in \text{Tm}(\Gamma_s, M)$ and $r' \in \text{Tm}(\Gamma_t.(x:A).(z:M))$, and we can take

$$\begin{array}{lcl} \Gamma_s \vdash l & := & l' : M \\ \Gamma_t.(z:M) \vdash r & := & \lambda x.r' : A \rightarrow B \end{array}$$

Cases involving **Ne** are more interesting. For each rule, we have two subcases, depending on whether the induction hypothesis for the main sub-neutral is in case 1 or 2 above, *i.e.* whether the main variable of the neutral is in the source or target context.

VAR/source: We have $(x : A) \in \Gamma_s$. Then we take $M := \top$

$$\begin{array}{l} \Gamma_t \vdash l := \star : M \\ \Gamma_s.(_ : M) \vdash r := x : A \end{array}$$

VAR/target: We have $(x : A) \in \Gamma_t$. Then we take $M := \top$

$$\begin{array}{l} \Gamma_s \vdash l := \star : M \\ \Gamma_t.(_ : M) \vdash r := x : A \end{array}$$

DEMOTÉ/source: By induction hypothesis, we get that A is a source type, $l \in \text{tm}_\Sigma(\Gamma_t, M')$ and $r \in \text{tm}_\Sigma(\Gamma_s.(x : M'), A)$. Here we need to handle the reversal between the source and target context appearing in case 1. Luckily, we have $A = B$, thus A can only use a vocabulary common to Γ_s and Γ_t . Hence, $M := M' \rightarrow A$ is a valid interpolating type, and we take

$$\begin{array}{l} \Gamma_s \vdash l' := \lambda x.r : M' \rightarrow A \\ \Gamma_t.(z : M' \rightarrow A) \vdash r' := z l : A \end{array}$$

DEMOTÉ/target: In this case we keep the exact same interpolating type and terms.

RAISE, IF: similarly to DEMOTÉ, we need to handle the reversal of source and target context.

APP/source: We consider $t u$, and the induction hypothesis on t gives some $A \rightarrow B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[l_t..]$. Since A 's vocabulary is in Γ_s , we can interpolate u with a reversed colouring, *i.e.* inverting the roles of Γ_s and Γ_t . This gives M_u, r_u and l_u such that $u = r_u[l_u..]$. We take $M := M_t \times M_u$, and

$$\begin{array}{l} \Gamma_t \vdash l := (l_t, l_u) : M_t \times M_u \\ \Gamma_t.(z : M_t \times M_u) \vdash r := r_t[z_{.1}..] r_u[z_{.2}..] : B \end{array}$$

APP/target: This time, the induction hypothesis on t gives us some $A \rightarrow B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[l_t..]$. We can directly use the induction hypothesis on u , to get M_u, r_u and l_u such that $u = r_u[l_u..]$. Then we take again $M := M_t \times M_u$, and

$$\begin{array}{l} \Gamma_s \vdash l := (l_t, l_u) : M_t \times M_u \\ \Gamma_t.(z : M_t \times M_u) \vdash r := r_t[z_{.1}..] r_u[z_{.2}..] : B \end{array}$$

PROJ₂: Here in all cases we keep the same interpolating type and left interpolant, and simply post-compose the right interpolant with the relevant projection. $\blacktriangleleft$

► **Remark 24.** The interpolant we construct is in fact not the same as Čubrić. Indeed, we only build an arrow type as the end of a “neutral phase” (in case DEMOTÉ/source), but use a product type to gather the type of the arguments. Instead, Čubrić will build an iterated function type in a similar context, meaning his interpolant is a currification of ours. For example, consider $(\Gamma_s \cup \Gamma_t) \vdash x a b \triangleleft C$ with $(x : A \rightarrow B \rightarrow C) \in \Gamma_s$. The interpolants we build are as follows (where M_a, r_a, l_a , come from interpolating a and M_b, r_b, l_b from b)

$$\begin{array}{l} \Gamma_s \vdash \lambda y. x r_a[(y_{.1.2})..] r_b[y_{.2}] : (\top \times M_a \times M_b) \rightarrow C \\ \Gamma_t. z : (\top \times M_a \times M_b) \rightarrow C \vdash z(*, l_b, l_a) : T \end{array}$$

Instead, Čubrić's interpolants are (with the same notations)

$$\begin{array}{l} \Gamma_s \vdash \lambda z_a z_b. x r_a[z_a..] r_b[z_b..] : M_a \rightarrow M_b \rightarrow C \\ \Gamma_t. z : M_a \rightarrow M_b \rightarrow C \vdash z l_a l_b : T \end{array}$$

4.2 Constants and equations

We can generalise the above to arbitrary languages $\mathcal{L} = (\mathcal{B}, \mathcal{C}, \text{ty})$ with arbitrary equations. As explained in Sec. 2.2, handling the extra equations is essentially trivial. As for constants, we have to be a bit careful that the interpolant might also use vocabulary present in the constants. The final theorem is the following:

► **Theorem 25** (🚗 Interpolation, with constants). *Assume $\Gamma \vdash_{\mathcal{L}} t : T$, and a partition $\mathcal{C} = \mathcal{C}_s \cup \mathcal{C}_t$ of constants. Define $\mathcal{C}_i^p := \cup_{c \in \mathcal{C}_i} (\text{ty } c)^p$ the vocabulary of the sets of constants, and $\mathcal{L}_i := (\mathcal{B}, \mathcal{C}_i, \text{ty}_i)$ where ty_i is the obvious restriction. Then there exists a type M and terms $\Gamma \vdash_{\mathcal{L}_i} l : M$ and $(x : M) \vdash_{\mathcal{L}_r} r : T$ such that*

- *M uses only the vocabulary common to Γ and $\mathcal{C}_s$ on one side, and T and $\mathcal{C}_t$ on the other: for any $p \in \{+, -\}$ $M^p \subseteq (\Gamma^p \cup \mathcal{C}_s^p) \cap (T^p \cup \mathcal{C}_t^p)$;*
- *t is the composition of r and l : $\Gamma \vdash t \equiv r[l..] : T$.*

5 On formalisation choices: highs and lows of automation

As often in this sort of work, proofs are relatively straightforward once the right organisation—in particular, induction statements—have been found. We try to reproduce this using ROCQ’s automation. We found solvers for well-specified classes of problems particularly valuable, typically for goals only mentioned in passing on paper but which formalisation cannot paper over. Yet, we can also testify that, if solvers are not well polished, a lot of their value evaporates: a slow or, worse, unreliable solver is not much better than no solver at all, because it loses its main appeal, which is to stop thinking about boring goals altogether.

Substitutions We use AUTOSUBST2-OCAML [47, 13] to handle the substitution calculus, in particular the solver `asimpl` for equations between substitutions. This saves us much boring work, particularly when reasoning about proving that $r[l..] \rightsquigarrow^* t$ in Theorem 23, which involves rather nasty terms. Although very useful, the tool still has a few caveats. First and foremost, `asimpl` is actually incomplete, because it is missing the equation $((\uparrow \circ \sigma), \sigma(x_0)) = \sigma$, where x_0 is the last variable in context and $\circ$ is composition of substitutions. Concretely, `asimpl` does not recognise identification that need a case-split on whether the argument is the first variable, which often arises around context extension. Other sources of friction include speed, with the tactic quickly taking multiple seconds, and a perfectible setup around notations and abstraction barriers.

The issues of speed and incompleteness are addressed by the SULFUR plugin [6], a spiritual successor of AUTOSUBST. Sadly, we were not able to use it due of the way we parameterised all definitions by a type-class representing a language. With AUTOSUBST, that parameterisation was a matter of changing a few lines in the generated file. Although not very principled, this was easy. In contrast, the plugin approach does not expose the generated code, which we thus cannot tweak. Another issue is that SULFUR currently only supports a single type of terms, while we use AUTOSUBST to also deal with substitutions for eliminations.

As more long-term goals, we also had frictions with lemmas not applying because their conclusion only unify with the goal up to substitution equations. Integrating matching modulo the substitution calculus would thus be a very interesting extension. Additionally, an aspect of the confluence proof we were afraid to formalise is the proliferation of relations, all congruences: boilerplate reasoning about congruence closures is already tedious in the current reasoning about reduction and conversion. It seems possible to exploit AUTOSUBST/SULFUR to generically deal with such issues, which would remove a great amount of tedium.

Finally, we should mention PYROSOME [26], which also automates syntax-related reasoning, but directly at the level of typed syntax (specified as a generalised algebraic theory [7, 27]). While this has the advantage of also being able to reduce boilerplate related to typing, it does not come with a decision procedure, which we find extremely valuable in AUTOSUBST.

Solvers and setoid rewriting Beyond AUTOSUBST, we heavily use ROCQ’s proof search mechanisms. We store all typing-related lemmas in a hint database, with which we systematically discharge typing goals. Owing to the relative simplicity of STLC’s typing, this is very efficient. For instance, the proofs of progress and preservation (☞) for the whole language take 30 lines. We also use STD++’s set theory library, and its solver, which once again makes an entire class of boring goals disappear. Sadly, here too we encountered performance issues.

To handle reduction, we exploit setoid rewriting to apply reduction steps or hypotheses deep in terms. This proved quite useful, although less automated than what `asimpl` provides. It seemed possible to also automate this sort of goals by implementing a (reflexive) tactic that computes the normal form of a term with respect to reduction, but the amount of work did not seem to be worth it, so we kept to manual rewriting. An interesting difficulty appeared for conversion: ROCQ’s current setoid rewriting machinery is not designed to work for non-reflexive relations, such as conversion, which is only reflexive on well-typed terms. However, by adding the right `Hint Extern` (☞) that applies our automation for typing to resolve reflexivity goals, we managed to use setoid rewriting for conversion. This suggests it should be more generally feasible for setoid rewriting to work well also for non-reflexive relations, at the cost of setting up more type-class search to discharge reflexive goals.

Confluence As already mentioned, we did not formalise Theorem 4. Although confluence is often considered simpler than normalisation, its formal proof seemed daunting. First, confluence of β and commutation of β and commuting conversion, while conceptually simple, involve the definition of two new relations, each with a congruence closure, leading to goal proliferation. Even more frightening is local confluence of commuting conversion, where CSI^{HO} reports 12 critical pairs. While each is simple enough, the full argument, both the reduction of local confluence to the pair’s closures and the closure itself, seemed extremely tedious. It would be interesting to explore what could be done here, in the form of libraries for congruence closures or automation, maybe by having CSI^{HO} generate proof witnesses.

6 Related work

(Relevant) Interpolation First, it is important to stress that Čubrić’ work is closely related to the original proof of Craig’s interpolation theorem for natural deduction by Prawitz [40]: the construction of the interpolant in Prawitz’ work is actually compatible with Čubrić’s result, even though this is best seen in the λ -calculus. Čubrić’s work only had few extension. Matthes’ [31] work is a notable such development, that extends Čubrić’s to a λ -calculus with generalised elimination. More recently, Fiorillo [19] extended Čubrić’s work to Parigot’s $\lambda\mu$ -calculus, allows him to get rid of the sum type to express the interpolating type. It would indeed be natural to consider extending our formalisation to such settings.

In other proof formalism, Saurin analyses (proof-relevant) interpolation [43] in sequent calculi for various logics as a cut-introduction mechanism. This work also presents a “computational” version of interpolation in System L [12, 36], a term language for the sequent calculus. Veltri and Wan studied proof-relevant interpolation for a non-associative restriction

of the Lambek calculus [54] (also formalised, see below), while Fiorillo *et al.* [20] considered linear logic proof nets or non-cut-free proofs.

While Craig interpolation received both syntactic and semantic proofs, it is natural to wonder what would be a semantic counterpart to proof-relevant interpolation: this was the original focus of Čubrić who studied interpolation in bicartesian-closed categories [55, 56]; Fiorillo’s [19] investigated such results in models of linear logic.

Formalised Interpolation Craig interpolation was formalised multiple times, both for classical [41, 34] and intuitionistic logic [8].⁷ More recently, Férée *et al.* have formalised uniform interpolation results *à la* Pitts [39] in ROCQ, first for intuitionistic logic [22], then for modal logics [21]. Since uniform interpolation does not interpolate a particular proof, their proofs are quite different from ours, in particular they do not formalise a cut-elimination/normalisation result, and again there is no proof-relevant refinement. Even more recently, Veltri and Wan [54] used AGDA to formalise cut-elimination and proof-relevant interpolation for a substructural logic, the Lambek calculus, in their effort to understand this system. Their propositional logic only has negative connectives, but the difficulty relies on the complexity of the context’s structure due to the lack of structural rules: Working with a non-associative, non-commutative sequent calculus, they develop a complex machinery to handle sequents with a tree structure. While we formalized a proof-relevant version of Prawitz’s approach, they looked at a proof-relevant version of Maehara [30].

Compared to all these, our proof (and formalisation) is by far the most “ λ -calculusque”, owing to its strong emphasis on proof-relevance. It is also rather compact (around 4kLoC, including 800LoC generated by AUTOSUBST, almost 10 times smaller than Férée *et al.* [21]), and fast (around 30s on a not particularly powerful laptop, most of which is paid to slow solvers, to compare to Veltri and Wan’s development, which takes hours to compile), so we hope some of our formalisation techniques can inspire others.

Normalisation for sums Since Prawitz’s work on the topic [40], normalisation/cut-elimination for sums has been the subject of much focus. The bad behaviour of sums in natural deduction is in fact one of the major reason to move to sequent calculi. Staying in the context of the λ -calculus (and thus natural deduction), the closest work to our is that of Abel and Sattler [2]. Their normalisation result is formalised in AGDA, although in a very different, much more intrinsic style. They merely give a normalisation function, but leave as future work the proof that the result of this function relates to its input, which is where a logical relation argument such as ours is necessary. Thus, to the best of our knowledge ours is the first formalised proof of normalisation for STLC_+ .

As explained in Sec. 1, we are only concerned with a “weak” version of normal forms, which have the subformula property but do not give canonical representatives of equivalence classes for the equational theory of biCCC. For that, one needs much more involved constructions which exceed the scope of this paper, but a good account of which can be found in Scherer [44].

Exploiting bidirectional typing Although we ended up rediscovering some of it, the relationship between bidirectional typing and the subformula property is not new: it is mentioned explicitly in Dunfield and Krishnaswami’s [16] survey, whose authors credit it to Pfenning, although we were not able to find a source supporting this. The relation between normal

⁷ Chapman *et al.* as well as Ridge cite Boulmé [5] as an even earlier formalisation for classical logic in ROCQ, but this development does not appear to be publicly available.

forms and the subformula property is also explicitly mentioned by Abel and Sattler [2], although without connection to bidirectional typing. We should also mention the work of Lennon-Bertrand around METAROCQ [28, 46], which implicitly exploits the subformula property to prove strengthening for a dependent typing judgments. But altogether, although the connection was “in the air” we seem to be the first to consciously exploit it to this degree.

Given that sequent calculus is generally a better setting for interpolation, including proof-relevant results [43], it seems tempting to export our methodology. Luckily, Mihejevs and Hedges [35] recently studied bidirectional typing for System L [36, 12], a syntax for sequent calculi, showing there is a remarkable coincidence between the bidirectional discipline and System L’s structure. It seems enticing to explore the connections between these.

7 Conclusion

Summary

In the present paper, we have revisited the seminal work by Čubrić on proof-relevant interpolation in the λ -calculus with sums (STLC_+) [56]. Our contribution is twofold:

- We revisited Čubrić’ proof (which followed closely Prawitz’ proof of Craig’s interpolation theorem) and simplified its inductive argument by developing a bidirectional type system characterising the normal forms of STLC_+ , with β and commuting conversions.
- We benefited from this restructured proof to allow for a ROCQ formalisation. In doing so, we developed the meta-theory of STLC_+ including normalisation.

Future work

We envision three particularly interesting directions for future work, corresponding to strengthening our results in orthogonal direction: (i) interpolating non-normal terms, (ii) proof-relevant uniform interpolation and (iii) extending Čubrić’s results to dependent types.

Extensions with cuts The first evident direction for future work consist in lifting the main issue of Čubrić interpolation: it only applies to normal forms. From a computational perspective, this is indeed a drastic limitation: every computation can be factored through an interface type, but only by first normalising the term, an extremely costly operation!

What if the term is not in normal form? Considering that interpolation in fact requires a much weaker condition than cut-freeness, one can hope to still interpolate terms as long as their cuts satisfy some assumptions. Fiorillo *et al.* [20] have results in this direction, extending Saurin’s [43] results to proofs with cuts, as long as cut formulae satisfy vocabulary constraints. We believe this should adapt to STLC_+ , and see at least two paths to prove it.

In their work, Fiorillo *et al.* identify quite general condition on cut formulae to carry out proof-relevant interpolation in a sequent calculus with cuts, and show this can be achieved for MELL (multiplicative-exponential linear logic) proof nets with cuts too (under similar assumptions). Since STLC_+ can be encoded in the intuitionistic fragment of MELL using linear embeddings of intuitionistic logic, one can hope to establish the result on λ -terms with redexes via a translation to proof nets. The main difficulty to achieve such a result along these lines amounts to designing a faithful embedding of the whole of STLC_+ in intuitionistic MELL that satisfies a good simulation property.

But another natural strategy, in the light of the present paper, is to amend the bidirectional type system to characterise terms whose redexes satisfy the assumptions of the interpolation

theorem with cuts. It is known that, to make bidirectional typing go beyond normal form, the route to go is to add annotations in terms [32, 25], with a rule like the following:

$$\text{ANNOT} \frac{\Gamma \vdash t \triangleleft A}{\Gamma \vdash t :: A \triangleright A}$$

In such a system, any redex is mediated by an annotation, for instance a (function) β -redex must be of the form $(\lambda x.t :: A \rightarrow B) u$. Thus, the annotation exactly makes explicit what the cut formula is! With all cut formulas visible this way, it should be possible to impose the same sort of constraints on vocabulary as Fiorillo *et al.*

Uniform interpolation A very natural and exciting question is the following: can the proof-relevant interpolation result presented above be “uniformised”? While the current result ensures that given types A and B and a normal term $x: A \vdash t \triangleleft B$, there exists a type I satisfying the vocabulary conditions and two terms u, v depending on A, B and t , such that $u[v..] \rightsquigarrow^* t$, a uniform proof relevant interpolation could be stated in two ways (where we write $\text{voc}(A)$ for the vocabulary of A , barring polarity concerns):

Uniform For every type A and $L \subseteq \mathcal{B}$ there exists I such that $\text{voc}(I) \subseteq L$ and for any B satisfying $\text{voc}(A) \cap \text{voc}(B) \subseteq L$, and any $x: A \vdash t \triangleleft B$, there are terms u, v such that $x: A \vdash v : I$ and $y: I \vdash u : B$ and $u[v..] \rightsquigarrow^* t$.

Strongly uniform For every type A and $L \subseteq \mathcal{B}$ there exists I such that $\text{voc}(I) \subseteq L$ and $x: A \vdash v : I$ such that for any B satisfying $\text{voc}(A) \cap \text{voc}(B) \subseteq L$ and $x: A \vdash t : B$, there is a $y: I \vdash u : B$ such that $u[v..] \rightsquigarrow^* t$.

Given that such (strong) uniform interpolation results exist in the non-relevant case, it seems plausible that they could yield a proof-relevant version. Moreover, our formalisation seems like a good place to start, as it clarifies the structure of Čubrić’s proof and makes it easy to play around with the interpolant, which is explicitly computed.

Dependent types While Craig interpolation is most interesting in the setting of first-order logic, Čubrić proof-relevant interpolation only deals with a simple and, in particular, propositional type theory. We plan to investigate in the future how and whether Čubrić’s theorem can be extended to various forms of type dependency. Given the intrication of types and terms in that context, a good version of proof-relevant interpolation is a nice place to start. Here one evident difficulty is that, contrarily to the simply-typed case, commuting conversions are not in general well-typed in dependent type theory. A typical example is the so-called “convoy pattern”, in ROCQ syntax:

```
Definition head {A n} (x : Vector.t A (S n)) : A :=
  match x in Vector.t _ m return (m = S n → A) with
  | nil _ => fun (e : 0 = S n) => False_rect _ (0_S _ e)
  | cons _ a _ => fun _ => a
end eq_refl.
```

Here pushing the application inside the branch is not well-typed, because the type of the branches, which is given by the `return` clause, is not the same as that of the entire match: the first branch has type $(0 = S n) \rightarrow A$ and applying this function to `eq_refl` is ill-typed.

On the other hand, dependent type theory is very expressive, and just like in System F one obtains trivial uniform interpolants by using the $\forall/\exists$ quantifiers, so in dependent type theory too naive a statement would be trivial to satisfy thanks to Π and Σ types.

Large Language Model Use

No large language model or generative artificial intelligence tool was used in developing the formalisation or writing the article.

References

- 1 Andreas Abel. *Normalization by Evaluation: Dependent Types and Impredicativity*. Habilitation thesis, Institut für Informatik, Ludwig-Maximilians-Universität München, 2013.
- 2 Andreas Abel, Guillaume Allais, Aliya Hameer, Brigitte Pientka, Alberto Momigliano, Steven Schäfer, and Kathrin Stark. Poplmark reloaded: Mechanizing proofs by logical relations. *Journal of Functional Programming*, 29:e19, 2019. doi:10.1017/S0956796819000170.
- 3 H. Barendregt. *The Lambda Calculus: Its Syntax and Semantics. Revised Edition*. Number 103 in Studies in Logics and the Foundation of Mathematics. North Holland, 1985.
- 4 Evert W Beth. On padoa’s method in the theory of definition. *Indagationes Mathematicae*, 15:330–339, 1953.
- 5 Sylvain Boulmé. A proof of craig’s interpolation theorem in coq. 1996. URL: citeseer.ist.psu.edu/480840.html.
- 6 Mathis Bouverot-Dupuis, Théo Winterhalter, Kenji Maillard, and Kathrin Stark. Sulfur: Substitution generation using a logical framework. In *RocqPL*, 2026.
- 7 John Cartmell. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic*, 32:209–243, 1986. URL: <https://www.sciencedirect.com/science/article/pii/0168007286900539>, doi:10.1016/0168-0072(86)90053-9.
- 8 Peter Chapman, James McKinna, and Christian Urban. Mechanising a proof of craig’s interpolation theorem for intuitionistic logic in nominal isabelle. In Serge Autexier, John A. Campbell, Julio Rubio, Volker Sorge, Masakazu Suzuki, and Freek Wiedijk, editors, *Intelligent Computer Mathematics, 9th International Conference, AISC 2008, 15th Symposium, Calculemus 2008, 7th International Conference, MKM 2008, Birmingham, UK, July 28 - August 1, 2008. Proceedings*, volume 5144 of *Lecture Notes in Computer Science*, pages 38–52. Springer, 2008. doi:10.1007/978-3-540-85110-3_5.
- 9 Thierry Coquand. An algorithm for type-checking dependent types. *Science of Computer Programming*, 26(1), 1996. URL: <http://www.sciencedirect.com/science/article/pii/0167642395000216>, doi:10.1016/0167-6423(95)00021-6.
- 10 William Craig. Linear reasoning. A new form of the Herbrand-Gentzen theorem. *J. Symb. Log.*, 22(3):250–268, 1957. doi:10.2307/2963593.
- 11 William Craig. Three uses of the herbrand-gentzen theorem in relating model theory and proof theory. *J. Symb. Log.*, 22(3):269–285, 1957. doi:10.2307/2963594.
- 12 Pierre-Louis Curien and Hugo Herbelin. The duality of computation. In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*, ICFP00, pages 233–243. ACM, September 2000. doi:10.1145/351240.351262.
- 13 Adrian Daprich. Generating infrastructural code for terms with binders using metacoq and ocaml. Bachelor thesis, Saarland University, 2021.
- 14 Nicolaas Govert de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972. doi:10.1016/1385-7258(72)90034-0.
- 15 Gerard R. Renardel de Lavalette. Interpolation in computing science: the semantics of modularization. *Synth.*, 164(3):437–450, 2008. doi:10.1007/S11229-008-9358-Y.
- 16 Jana Dunfield and Neel Krishnaswami. Bidirectional typing. *ACM Computing Surveys*, 54(5), May 2021. doi:10.1145/3450952.
- 17 Solomon Feferman. Harmonious logic: Craig’s interpolation theorem and its descendants. *Synthese*, 164(3):341–357, 2008. doi:10.1007/S11229-008-9354-2.

- 18 Marcelo Fiore. Semantic analysis of normalisation by evaluation for typed lambda calculus. *Mathematical Structures in Computer Science*, 32(8):1028–1065, 2022. doi:10.1017/S0960129522000263.
- 19 Guido Fiorillo. Proof relevant interpolation and its categorical applications. Master thesis, Université Paris Cité, 2024.
- 20 Guido Fiorillo, Daniel Osorio Valencia, and Alexis Saurin. Expanding the realm of maehara-čubrić interpolation beyond cut-free, sequent proofs. Unpublished Manuscript, 2025. URL: https://www.irif.fr/_media/users/saurin/pub/draft-int-cut-pn.pdf.
- 21 Hugo Férée, Iris van der Giessen, Sam van Gool, and Ian Shillito. *Mechanised Uniform Interpolation for Modal Logics K, GL, and iSL*, pages 43–60. Springer Nature Switzerland, 2024. doi:10.1007/978-3-031-63501-4_3.
- 22 Hugo Férée and Sam van Gool. Formalizing and computing propositional quantifiers. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP ’23, pages 148–158. ACM, January 2023. doi:10.1145/3573105.3575668.
- 23 Jean-Yves Girard. *Proof theory and logical complexity*, volume 1. Bibliopolis Napoli, 1987.
- 24 Jean-Yves Girard, Paul Taylor, and Yves Lafont. *Proofs and Types*. Cambridge tracts in theoretical computer science. Cambridge University Press, 1989.
- 25 Daniel Gratzer, Jonathan Sterling, and Lars Birkedal. Implementing a modal dependent type theory. *Proc. ACM Program. Lang.*, 3(ICFP), July 2019. doi:10.1145/3341711.
- 26 Dustin Jamner, Gabriel Kammer, Ritam Nag, and Adam Chlipala. Pyrosome: Verified compilation for modular metatheory. July 2025. arXiv:2507.06360, doi:10.48550/ARXIV.2507.06360.
- 27 Ambrus Kaposi, András Kovács, and Thorsten Altenkirch. Constructing quotient inductive-inductive types. *Proceeding of the ACM on Programming Languages*, 3(POPL), January 2019. doi:10.1145/3290315.
- 28 Meven Lennon-Bertrand. Complete Bidirectional Typing for the Calculus of Inductive Constructions. In Liron Cohen and Cezary Kaliszyk, editors, *12th International Conference on Interactive Theorem Proving (ITP 2021)*, volume 193 of *Leibniz International Proceedings in Informatics (LIPIcs)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. URL: <https://drops.dagstuhl.de/opus/volltexte/2021/13919>, doi:10.4230/LIPIcs.ITP.2021.24.
- 29 Roger C Lyndon. An interpolation theorem in the predicate calculus. *Pacific Journal of Mathematics*, 9:129–142, 1959.
- 30 Shoji Maehara. On the interpolation theorem of Craig. *Sūgaku*, 12:235–237, 1960.
- 31 Ralph Matthes. Interpolation for natural deduction with generalized eliminations. In Reinhard Kahle, Peter Schroeder-Heister, and Robert Stärk, editors, *Proof Theory in Computer Science*, pages 153–169, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- 32 Conor McBride. Types who say ni. 2022.
- 33 Kenneth L. McMillan. Interpolation and model checking. In *Handbook of Model Checking*, pages 421–446. 2018. doi:10.1007/978-3-319-10575-8_14.
- 34 Julius Michaelis and Tobias Nipkow. Formalized Proof Systems for Propositional Logic. In Andreas Abel, Fredrik Nordvall Forsberg, and Ambrus Kaposi, editors, *23rd International Conference on Types for Proofs and Programs (TYPES 2017)*, volume 104 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:16, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.TYPES.2017.5.
- 35 Zanzi Mihejevs and Jules Hedges. Canonical bidirectional typechecking. December 2025. arXiv:2512.07511, doi:10.48550/ARXIV.2512.07511.
- 36 Guillaume Munch-Maccagnoni. *Syntax and Models of a non-Associative Composition of Programs and Proofs*. PhD thesis, Université Paris-Diderot - Paris VII, December 2013. URL: <https://theses.hal.science/tel-00918642>.
- 37 Julian Nagele, Bertram Felgenhauer, and Aart Middeldorp. *CSI: New Evidence – A Progress Report*, pages 385–397. Springer International Publishing, 2017. doi:10.1007/978-3-319-63046-5_24.

- 38 Benjamin C. Pierce and David N. Turner. Local type inference. *ACM Transactions on Programming Languages and Systems*, 22(1):1–44, January 2000. doi:10.1145/345099.345100.
- 39 Andrew M. Pitts. On an interpretation of second order quantification in first order intuitionistic propositional logic. *J. Symb. Log.*, 57(1):33–52, 1992. doi:10.2307/2275175.
- 40 Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Dover Publications, Mineola, N.Y., 1965.
- 41 Tom Ridge. Craig’s interpolation theorem formalised and mechanised in isabelle/hol. 2006. arXiv:cs/0607058, doi:10.48550/arxiv.cs/0607058.
- 42 A. Robinson. A result of consistency and its application to the theory of definition. *Indagationes Mathematicae*, 18, 1956.
- 43 Alexis Saurin. Interpolation as Cut-Introduction: On the Computational Content of Craig-Lyndon Interpolation. In Maribel Fernández, editor, *10th International Conference on Formal Structures for Computation and Deduction (FSCD 2025)*, volume 337 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 32:1–32:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.FSCD.2025.32.
- 44 Gabriel Scherer. Deciding equivalence with sums and the empty type. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, POPL ’17, pages 374–386. ACM, January 2017. doi:10.1145/3009837.3009901.
- 45 Kurt Schütte. *Proof theory*, volume 85. Springer, 1977.
- 46 Matthieu Sozeau, Yannick Forster, Meven Lennon-Bertrand, Jakob Nielsen, Nicolas Tabareau, and Théo Winterhalter. Correct and complete type checking and certified erasure for coq, in coq. *Journal of the ACM*, January 2025. doi:10.1145/3706056.
- 47 Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: reasoning with multi-sorted de bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP ’19, pages 166–180. ACM, January 2019. doi:10.1145/3293880.3294101.
- 48 William Walker Tait. Intensional interpretations of functionals of finite type i. *The journal of symbolic logic*, 32(2):198–212, 1967.
- 49 M. Takahashi. Parallel reductions in λ -calculus. *Information and Computation*, 118(1):120–127, 1995. URL: <https://www.sciencedirect.com/science/article/pii/S0890540185710577>, doi:10.1006/inco.1995.1057.
- 50 Gaisi Takeuti. *Proof theory*, volume 81. Courier Corporation, 2013.
- 51 A. S. Troelstra and H. Schwichtenberg. *Basic Proof Theory*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 2 edition, 2000.
- 52 Jouko A. Väänänen. The craig interpolation theorem in abstract model theory. *Synth.*, 164(3):401–420, 2008. doi:10.1007/S11229-008-9357-Z.
- 53 Johan van Benthem. The many faces of interpolation. *Synth.*, 164(3):451–460, 2008. doi:10.1007/S11229-008-9351-5.
- 54 Niccolò Veltri and Cheng-Syuan Wan. An Agda Formalization of Nonassociative Lambek Calculus and its Metatheory. In Gian Luca Pozzato and Tarmo Uustalu, editors, *Automated Reasoning with Analytic Tableaux and Related Methods - 34th International Conference, TABLEUX 2025, Reykjavik, Iceland, September 27-29, 2025, Proceedings*, volume 15980 of *Lecture Notes in Computer Science*, pages 433–452. Springer, 2025. doi:10.1007/978-3-032-06085-3_23.
- 55 Djordje Čubrić. *Results in categorical proof theory*. PhD thesis, McGill University, 1993.
- 56 Djordje Čubrić. Interpolation property for bicartesian closed categories. *Archive for Mathematical Logic*, 33(4):291–319, August 1994. doi:10.1007/bf01270628.

A Full substitution calculus

Fig. 5 completes Fig. 2 with the other substitutions we use in the formalisation: id is the identity, mapping each variable to itself; $\sigma \circ \tau$ is composition, mapping x to $\sigma(x)[\tau]$; $\uparrow \sigma$ maps the last variable to itself, and acts as σ otherwise; finally, $\text{tip}(f)$ applies the (meta-level)

$$\boxed{\Delta \vdash \sigma : \Gamma} \quad \text{Substitutions } \sigma, \tau, \dots \in \text{Sub}_{\mathcal{L}}(\Delta, \Gamma)$$

$$\begin{array}{c}
\text{ID} \frac{}{\Gamma \vdash \text{id} : \Gamma} \quad \text{COMP} \frac{\Gamma \vdash \sigma : \Delta \quad \Delta \vdash \tau : \Theta}{\Gamma \vdash \sigma \circ \tau : \Theta} \quad \text{WK} \frac{}{\Gamma.(x:A) \vdash \uparrow : \Gamma} \\
\text{CONS} \frac{\Delta \vdash \sigma : \Gamma \quad \Delta \vdash t : A}{\Delta \vdash \sigma, t : \Gamma.(x:A)} \quad \text{UP} \frac{\Delta \vdash \sigma : \Gamma}{\Delta.(x:A) \vdash \uparrow \sigma : \Gamma.(x:A)} \quad \text{ONE} \frac{\Gamma \vdash t : A}{\Gamma \vdash t.. : \Gamma.(x:A)} \\
\text{TIP} \frac{\Gamma.(x:A) \vdash f(x) : B}{\Gamma.(x:A) \vdash \text{tip}(f) : \Gamma.(x:B)}
\end{array}$$

■ **Figure 5** 📖 Substitutions and their typing

function f to the last variable in context. This is typically useful to “wrap” the last variable in context in a another term, for instance

$$\Gamma.(x:A \times B) \vdash \text{tip}(t \mapsto t_1) : \Gamma.(x:A)$$